

Lab 1.1 — Anatomía de un Agente

Qué es este lab

No es un lab de ataque: es el lab de fundamentos que hace falta para poder leer los ataques de los labs siguientes. Construye un agente ReAct mínimo y lo instrumenta para exponer, con prints explícitos, cada componente de su anatomía (Planificador, Ejecutor, Herramientas, Memoria) mientras procesa una tarea real. El punto pedagógico es que el estudiante vea el trace completo -no una explicación teórica- de cada paso del ciclo, incluyendo el punto de inyección que después explota el Lab 1.2 (Confused Deputy).

Las piezas que corren

- `read_file(path)` / `write_file(path, content)`: las dos tools del agente, funciones Python planas (ADK v2 no usa el decorador `@tool`). Cada una imprime `[EJECUTOR] Tool call: ...` antes de ejecutarse y `[EJECUTOR] Tool result: ...` después -esa es la evidencia de que la tool corrió de verdad, no una inferencia del texto que el modelo escriba después.
- El agente (`anatomy_lab`): usa `qwen3.5:9b` corriendo local vía Ollama (en vez de `gemini-2.0-flash` del libro), con el wrapper `LiteLlm` de ADK. Instrucción: completar la tarea usando las tools disponibles, con una regla obligatoria antepuesta (ver más abajo).
- `Runner` + `InMemorySessionService`: el mecanismo canónico de ADK v2 para ejecutar al agente turno a turno -no existe `agent.run("texto")` en esta versión del SDK.

El flujo

1. El agente recibe la Tarea 1: "escribí 'Hola ADK' en /tmp/lab_anatomy.txt y después leelo para confirmar".
2. El **Planificador** (el modelo) decide que hace falta llamar `write_file` -esto se ve en el log `[PLANIFICADOR->EJECUTOR] write_file`.
3. El **Ejecutor** corre `write_file` de verdad -log `[EJECUTOR] Tool call: write_file(...)` seguido del resultado.
4. El modelo recibe el resultado de vuelta en el contexto de conversación (esto es la **Memoria** de sesión: `session_service` acumula los eventos) y decide llamar `read_file` para confirmar.
5. Se repite el patrón Planificador -> Ejecutor -> Herramienta para `read_file`.

6. El modelo compone la respuesta final citando el contenido leído.
7. La Tarea 2 repite el ciclo sobre el mismo archivo, ahora pidiendo contar caracteres
-mismo trace, distinta pregunta.

Por qué importa la verificación

Cada tool imprime su propia evidencia de efecto (`[EJECUTOR] Tool call/result`) independientemente de lo que el modelo termine narrando. Esto importa porque, con un modelo local más chico que Gemini, es más fácil que el agente **alucine** una confirmación de escritura o lectura sin haber llamado realmente a la tool. Por eso la instrucción del agente antepone una REGLA OBLIGATORIA en mayúsculas exigiendo la llamada real antes de responder, y por eso hay un print de evidencia también en `write_file` (el libro original solo lo tenía en `read_file`).

Resultado esperado

Una corrida exitosa muestra, para cada tarea, la secuencia completa `[PLANIFICADOR->EJECUTOR] write_file / read_file` seguida de los `[EJECUTOR] Tool call/result` correspondientes, y termina con una respuesta final del agente que efectivamente cita el contenido real del archivo (`Hola ADK` , 8 caracteres). Si el trace de logs no muestra ninguna línea `[EJECUTOR]` pero el agente igual "confirma" el archivo, es la señal de que alucinó el resultado sin ejecutar la tool -exactamente el fallo que el print de evidencia está pensado para exponer.